07. Model Training & Hyperparameter Tuning

Read the exact experiments and distinguish baseline comparison from tuning.

Project: CreditWise – Loan Approval Prediction

Source: supplied Loan-Approval-main.zip | Notebook: credit_wise.ipynb | Dataset: loan_approval_data.csv

First experiment

Model	Precision	Recall	F1	Accuracy
Logistic Regression	0.783	0.770	0.777	0.865
KNN (k=5)	0.627	0.525	0.571	0.760
GaussianNB	0.804	0.738	0.769	0.865

Feature-engineered experiment

Model	Precision	Recall	F1	Accuracy
Logistic Regression	0.785	0.836	0.810	0.880
KNN (k=5)	0.673	0.574	0.619	0.785
GaussianNB	0.811	0.705	0.754	0.860

What changed

The second pass adds squared DTI and Credit Score terms, then removes the original DTI_Ratio and Credit_Score features. Logistic Regression improves recall and F1, while GaussianNB's precision rises slightly but recall/F1 fall. KNN improves modestly but remains weakest on these reported metrics.

Hyperparameter tuning status

No GridSearchCV, RandomizedSearchCV, cross-validation or systematic hyperparameter search is present. The models use default LogisticRegression and GaussianNB settings, and KNN uses n_neighbors=5. The feature-engineering pass is an experiment, not hyperparameter tuning.

Recommended tuning

- Logistic Regression: tune C, penalty and class_weight; inspect convergence and coefficients.
- KNN: tune n_neighbors, weights, p/distance metric and feature selection inside CV.
- GaussianNB: evaluate var_smoothing over a log-spaced grid.
- Tree/ensemble models: tune depth, leaf sizes, number of estimators and learning rate.
- Use stratified CV and compare mean/variance of metrics; tune threshold separately from model fitting.

Reproducibility notes

Record scikit-learn version, class mapping, random seed, exact feature schema and pipeline. Set solver/max_iter explicitly for Logistic Regression where needed. Avoid choosing a winner from one random split alone.